```
################################################################################
# LAB DEMO: Classification using K-Nearest Neighbors (KNN) in R
# Course: Data Mining & Warehousing
# Dataset: iris (built-in) — 150 samples, 3 species, 4 numeric features
################################################################################

# ------------------------------------------------------------------------------
# 1. SETUP — install/load required packages
# ------------------------------------------------------------------------------
required_pkgs <- c("class", "caret", "ggplot2", "gmodels")
to_install <- required_pkgs[!(required_pkgs %in% installed.packages()[, "Package"])]
if (length(to_install)) install.packages(to_install)

library(class)    # knn()
library(caret)    # confusionMatrix(), createDataPartition()
library(ggplot2)  # visualization
library(gmodels)  # CrossTable() for a readable confusion matrix

# ------------------------------------------------------------------------------
# 2. EXPLORE THE DATA
# ------------------------------------------------------------------------------
data(iris)
str(iris)
summary(iris)
table(iris$Species)        # check class balance

# Quick visual: are the classes separable?
```

```
ggplot(iris, aes(Petal.Length, Petal.Width, color = Species)) +
  geom_point(size = 2) +
  labs(title = "Iris: Petal Length vs Petal Width by Species")

# ------------------------------------------------------------------------
# 3. WHY NORMALIZE?
# KNN uses distance (usually Euclidean), so features on larger numeric
# scales dominate the distance calculation unless we rescale everything
# to a comparable range, e.g. [0, 1].
# ------------------------------------------------------------------------
normalize <- function(x) (x - min(x)) / (max(x) - min(x))

iris_norm <- as.data.frame(lapply(iris[, 1:4], normalize))
summary(iris_norm)          # confirm all features now range 0-1

# ------------------------------------------------------------------------
# 4. TRAIN / TEST SPLIT (stratified, 70/30)
# ------------------------------------------------------------------------
set.seed(123)  # reproducibility — always set this in a lab demo
train_index <- createDataPartition(iris$Species, p = 0.7, list = FALSE)

train_data  <- iris_norm[train_index, ]
test_data   <- iris_norm[-train_index, ]
train_labels <- iris$Species[train_index]
test_labels  <- iris$Species[-train_index]

cat("Training rows:", nrow(train_data), " | Test rows:", nrow(test_data), "\n")
```

```
# ------------------------------------------------------------------------
# 5. FIT A KNN MODEL (start with k = 5)
# ------------------------------------------------------------------------
knn_pred <- knn(train = train_data,
                test  = test_data,
                cl    = train_labels,
                k     = 5)

# ------------------------------------------------------------------------
# 6. EVALUATE
# ------------------------------------------------------------------------
# Readable cross-tabulation (predicted vs actual)
CrossTable(x = test_labels, y = knn_pred, prop.chisq = FALSE)

# Formal confusion matrix + accuracy, kappa, per-class stats
confusionMatrix(knn_pred, test_labels)

# ------------------------------------------------------------------------
# 7. CHOOSING k — accuracy vs k (elbow-style plot)
# A small k is sensitive to noise (overfits); a large k over-smooths
# (underfits) and can blur the decision boundary between classes.
# ------------------------------------------------------------------------
k_values <- 1:20
accuracy <- sapply(k_values, function(k) {
  pred <- knn(train_data, test_data, train_labels, k = k)
  mean(pred == test_labels)
})
```

```
k_results <- data.frame(k = k_values, accuracy = accuracy)
print(k_results)

ggplot(k_results, aes(k, accuracy)) +
  geom_line(color = "steelblue") +
  geom_point(size = 2, color = "steelblue") +
  scale_x_continuous(breaks = k_values) +
  labs(title = "KNN Accuracy vs k", x = "k (number of neighbors)", y = "Test Accuracy")

best_k <- k_results$k[which.max(k_results$accuracy)]
cat("Best k on this split:", best_k, "\n")

# ------------------------------------------------------------------------
# 8. FINAL MODEL WITH BEST k
# ------------------------------------------------------------------------
final_pred <- knn(train_data, test_data, train_labels, k = best_k)
confusionMatrix(final_pred, test_labels)

# ------------------------------------------------------------------------
# 9. CLASSIFY A NEW / UNSEEN SAMPLE
# ------------------------------------------------------------------------
new_sample <- data.frame(Sepal.Length = 5.1, Sepal.Width = 3.5,
                         Petal.Length = 1.4, Petal.Width = 0.2)

# Normalize the new sample using the SAME min/max as the training data
mins <- sapply(iris[, 1:4], min)
maxs <- sapply(iris[, 1:4], max)
new_sample_norm <- as.data.frame(mapply(function(x, mn, mx) (x - mn) / (mx - mn),
```

```
                    new_sample, mins, maxs))


knn(train_data, new_sample_norm, train_labels, k = best_k)
```